

Cómo aprenden los LLMs por dentro

Manual complementario

Predicción · Entrenamiento · Pesos · Funciones de activación

Preguntas que responde este manual

- ¿Por qué se dice que los LLMs “predicen” si vos ya diste el texto?
- ¿Qué predican realmente? ¿En qué sentido “entienden”?
- ¿Cómo se entrenan estos modelos?
- ¿Qué valores toman los pesos al principio?
- ¿Quién define las funciones de activación?

Con teoría, ejemplos numéricos y diagramas.

Contenidos

Parte I	¿Qué predicen y cómo “entienden” los LLMs?	3
Parte II	Cómo se entrenan los LLMs	6
Parte III	Los pesos y las funciones de activación	12
Parte IV	Síntesis final y glosario rápido	18

PARTE I

¿Qué predicen y cómo “entienden” los LLMs?

1. La confusión de raíz

Cuando escribís un prompt y apretás enviar, tu intuición dice:

“Yo le di TODO el texto. ¿Qué está prediciendo? ¿Qué está entendiendo? Ya está todo escrito.”

Esa intuición mezcla dos cosas distintas: **la entrada** (lo que le diste) y **la salida** (lo que va a generar). El modelo **no predice tu texto**. Predice **lo que viene después de tu texto**.

2. Qué predicen realmente

Un LLM tiene una única habilidad matemática, aprendida durante el entrenamiento:

Dado un texto cualquiera, calcular qué token es el más probable que aparezca a continuación.

Nada más. Esa es toda la maquinaria.

Figura E. Qué es exactamente lo que un LLM predice



El modelo NO predice tu prompt. Predice lo que viene DESPUÉS de tu prompt.

Figura E. Qué es exactamente lo que un LLM predice.

Ejemplo mecánico

Le das: “La capital de Francia es”

El modelo mira ese texto y se hace una sola pregunta: **¿qué palabra tiene sentido acá abajo?** Devuelve una distribución de probabilidades sobre TODO su vocabulario (unas 50.000 a 100.000 opciones):

- “ París” → 89%
- “ una” → 3%
- “ Lyon” → 1%
- ...miles con probabilidad muy chica

Elige uno (normalmente el más probable) y lo agrega al final. Ahora el texto es “La capital de Francia es París”. Y **vuelve a hacer lo mismo**: ¿qué viene después de esto? Probable: “.”. Lo agrega. ¿Y después? Un token especial de “fin”. Termina.

Cada palabra que ves aparecer en pantalla, una tras otra, es literalmente el modelo repitiendo el ciclo:

mirar todo → elegir siguiente → agregarlo → mirar todo otra vez → ...

Esto se llama **generación autorregresiva**. Tu prompt es solo el punto de partida del texto que va creciendo.

3. Entonces, ¿qué es “entender”?

Acá hay que ser honesto: **“entender” es una palabra prestada del vocabulario humano** que aplicamos por comodidad. El modelo no entiende como vos entendés. No tiene conciencia, ni intención, ni sabe que está conversando.

Lo que sí hace, y que **se parece funcionalmente** al entender:

Durante el entrenamiento leyó miles de millones de textos. Aprendió que ciertas palabras aparecen en ciertos contextos. Que después de “El gato” es más probable “duerme” que “duermen”. Que después de “*Explicame la fotosíntesis*” viene una explicación didáctica, no una receta de cocina. Que después de “*Actuá como un profesor*” el estilo cambia.

Toda esa información quedó codificada en sus **pesos** (miles de millones de números ajustables). Cuando le das un prompt nuevo, esos pesos, combinados con el mecanismo de **atención** del Transformer, hacen que el modelo:

- Reconozca el **rol** que le pediste (profesor, poeta, programador).
- Detecte la **audiencia** (nivel básico, experto).
- Identifique la **tarea** (responder, listar, resumir, crear un documento).
- Ajuste el **tono y vocabulario** según el contexto.

Nada de eso es consciente. Es un patrón estadístico enorme que **se comporta como si entendiera**, porque fue entrenado con textos escritos por humanos que sí entendían.

Analogía útil

Imaginá a alguien que **jamás** estudió español pero leyó un billón de libros en español. Nunca aprendió la regla “los verbos concuerdan con el sujeto”. Pero después de tanto leer, sabe que “*los gatos duerme*” suena mal y “*los gatos duermen*” suena bien. No sabe por qué. Solo sabe qué combinaciones aparecen y cuáles no.

Un LLM es esa persona llevada al extremo. Cuando le pedís algo, no razona sobre reglas: pesa qué continuación es más “sonora” según todo lo que vio. Y como vio muchísimo, esas continuaciones son sofisticadas, coherentes, útiles... y a veces sorprendentemente parecidas a pensar.

Resumen para fijar

- Vos le das un **texto de entrada** (tu prompt).
- El modelo predice el **próximo token** que sigue DESPUÉS de ese texto.
- Lo agrega, y vuelve a predecir el siguiente. Y así, palabra a palabra.
- **“Entender” es una metáfora**: hay un patrón estadístico gigante que se comporta como comprensión, sin serlo en el sentido humano.

PARTE II

Cómo se entrenan los LLMs

4. La idea general del entrenamiento

Cuando decimos “entrenar” un modelo, estamos diciendo algo muy concreto:

Ajustar sus miles de millones de pesos (los números internos) hasta que aprenda a predecir bien.

Al principio, esos pesos son **aleatorios**. El modelo es tonto: si le das “*El cielo es*”, puede contestar cualquier cosa: “banana”, “verde”, “corriendo”. No sabe nada.

El entrenamiento es el proceso de mostrarle muchísimo texto y decirle: “*cuando veas esto, esperá que venga esto otro*”. Cada vez que se equivoca, se corrigen un poquito los pesos. Repetido miles de millones de veces, el modelo termina siendo bueno.

5. Las tres fases del entrenamiento moderno

Los LLMs actuales (ChatGPT, Claude, Gemini) no salen listos en un solo paso. Pasan por **tres fases encadenadas**, cada una con un objetivo distinto.

Figura 12. Las tres fases del entrenamiento de un LLM moderno

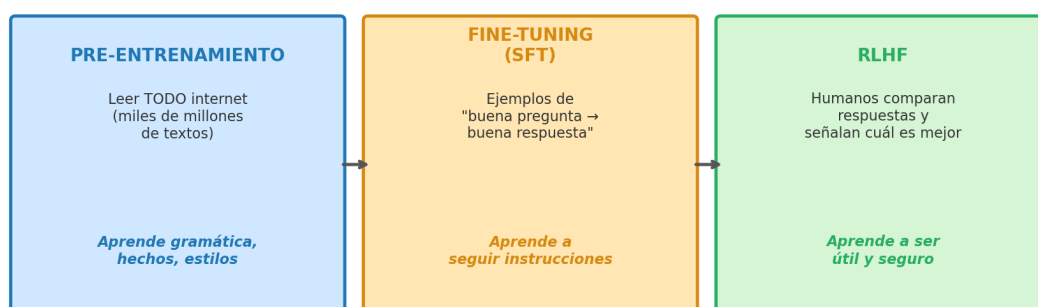


Figura 12. Las tres fases del entrenamiento de un LLM moderno.

5.1. Fase 1 — Pre-entrenamiento (la más grande y costosa)

Qué se le da al modelo: una cantidad enorme de texto:

- Toda Wikipedia.
- Millones de libros.
- Miles de millones de páginas web.
- Repositorios de código (GitHub).
- Artículos científicos, foros, subtítulos de videos.

En total: **billones de tokens** (con “b” de mil millones de millones).

Qué tarea le damos: predecir la siguiente palabra. Solo eso. Se toma un texto real, se le muestra al modelo un pedacito, y se le pide que adivine qué sigue.

Ejemplo mecánico

Frase del corpus: “*El sol sale por el este*”

El entrenamiento le da al modelo estos “ejercicios”:

Entrada: “El” → Respuesta correcta: “ sol”

Entrada: “El sol” → Respuesta correcta: “ sale”

Entrada: “El sol sale” → Respuesta correcta: “ por”

Entrada: “El sol sale por” → Respuesta correcta: “ el”

Entrada: “El sol sale por el” → Respuesta correcta: “ este”

Cada vez que el modelo predice mal, se calcula el error y un algoritmo llamado **backpropagation** (retropropagación) ajusta ligeramente los pesos para que la próxima vez se equivoque menos.

Hacé esto **billones de veces**, con billones de fragmentos distintos, y algo maravilloso pasa: para predecir bien la siguiente palabra, el modelo tiene que aprender —sin que nadie se lo diga explícitamente— gramática, hechos del mundo, cómo funcionan los diálogos, estilos de escritura, matemática básica, razonamiento, código. Todo **emerge** de la tarea simple de predecir la siguiente palabra sobre texto de calidad en cantidad brutal.

Costo: meses de cómputo en miles de GPUs. Decenas o cientos de millones de dólares. Consumo energético comparable al de un pueblo.

Resultado al final de la Fase 1: un modelo que sabe muchísimo, pero es raro de usar. Si le escribís “*Explicame la fotosíntesis*”, podría contestar “*Explicame la fotosíntesis en un párrafo. Tarea del alumno Juan Pérez, curso 2023...*” — completa como si fuera una página web, porque eso es todo lo que aprendió a hacer.

5.2. Fase 2 — Fine-tuning supervisado (SFT)

Ahora hay que enseñarle **modales**: a comportarse como un asistente que sigue instrucciones.

Qué se le da: un conjunto mucho más chico (decenas o cientos de miles) de ejemplos de **conversaciones ideales**, escritos por humanos entrenadores. Cada ejemplo tiene el formato:

Prompt del usuario: “Explicame la fotosíntesis en 3 oraciones simples.”

Respuesta ideal (escrita por un humano): “La fotosíntesis es el proceso por el que las plantas fabrican su alimento usando luz solar. Toman dióxido de carbono del aire y agua del suelo, y con la energía del sol los transforman en azúcares y oxígeno. Es la razón por la que las plantas son verdes y por la que hay oxígeno en la atmósfera.”

El modelo se ajusta para producir respuestas parecidas a las de los humanos. Ya no completa como una página web: ahora entiende que un prompt es una **instrucción a responder**, no un texto a continuar.

Es igual que la Fase 1 en su mecánica (ajustar pesos para predecir tokens), pero cambia radicalmente el **tipo de texto** que ve: en vez de internet crudo, ve conversaciones “modelo”.

Resultado tras Fase 2: un modelo útil y conversacional, pero todavía imperfecto. A veces contesta cosas peligrosas, otras se equivoca de tono, o da respuestas que suenan bien pero no son las mejores.

5.3. Fase 3 — RLHF (aprendizaje por refuerzo con retroalimentación humana)

La cereza del pastel. Cómo funciona, paso a paso:

1. Se toma un prompt.
2. El modelo (ya entrenado hasta la Fase 2) genera **varias respuestas distintas** al mismo prompt.
3. Un evaluador humano las **ordena de mejor a peor**: “esta es la más útil, esta la segunda, esta peor, esta muy mala”.
4. Con miles de comparaciones así, se entrena un **modelo de recompensa**: una red aparte que aprende a predecir qué respuesta preferiría un humano.
5. Ahora se usa ese modelo de recompensa como **juez automático**. El LLM principal genera respuestas, el juez las puntúa, y el LLM se ajusta para producir respuestas que reciban puntajes altos.

Es como enseñarle a un aprendiz mostrándole muchas veces “así se hace mejor, así peor”, hasta que internaliza el estilo.

Qué se logra con RLHF

- **Utilidad**: que la respuesta realmente resuelva lo que se pidió.
- **Honestidad**: que diga “no sé” cuando corresponde, en lugar de inventar.
- **Seguridad**: que rechace pedidos peligrosos, ilegales o dañinos.
- **Tono adecuado**: empático cuando toca, directo cuando toca, etc.

6. Los datos son todo

Un LLM **es**, en el fondo, un reflejo estadístico de los textos con los que se entrenó. Esto tiene consecuencias serias:

- **Sesgos de los datos** → **sesgos del modelo**. Si el corpus tiene prejuicios de género, ideológicos o culturales, el modelo los reproduce.
- **Lo que no está en los datos, el modelo no lo sabe**. Un LLM entrenado hasta 2023 no sabe qué pasó en 2024.
- **La calidad importa más que la cantidad**. Basura entra, basura sale.
- **Problemas legales**. Muchos datasets incluyen textos con derechos de autor: juicios y debates activos.

7. Cuánto cuesta todo esto

Para dimensionar la escala real de entrenar un LLM grande hoy:

Datos	Entre 1 y 15 billones de tokens (1 token $\approx$ 3/4 de palabra)
Cómputo	Miles a decenas de miles de GPUs en paralelo
Tiempo	De semanas a varios meses corridos
Costo	Entre 50 y 500 millones de dólares (modelo frontera)
Energía	Equivalente al consumo eléctrico anual de miles de hogares

Por eso solo un puñado de organizaciones en el mundo (OpenAI, Anthropic, Google, Meta, xAI, algunos labs chinos) entrena modelos frontera desde cero. La mayoría de los otros usan modelos ya entrenados y les hacen **fine-tuning propio**, mucho más barato.

PARTE III

Los pesos y las funciones de activación

8. Los pesos: valores iniciales y evolución

Ya sabemos que “entrenar” significa **ajustar pesos**. Pero ¿de dónde salen los primeros valores? ¿Quién los pone? ¿Cómo evolucionan?

8.1. Con qué valores empiezan los pesos

Al principio, antes de cualquier entrenamiento, **los pesos son números aleatorios**. Literalmente: se los inicializa con un generador de números al azar.

Pero no cualquier azar. Hay técnicas específicas para elegir esos valores iniciales, porque **si arrancás muy mal, la red no aprende nunca**. Las más famosas son:

- **Inicialización de Xavier/Glorot:** números aleatorios pequeños, con una distribución matemática cuidadosamente calculada según cuántas conexiones tenga cada neurona.
- **Inicialización de He:** parecida, pero pensada para funciones de activación específicas (ReLU).
- **Distribución normal:** valores centrados en cero, con una desviación pequeña (por ejemplo, entre $-0,1$ y $+0,1$).

Ejemplo concreto de valores iniciales

Supongamos una red pequeña con 3 pesos. Al inicializarla, podría empezar así:

$w_{11} = 0,0247$
 $w_{12} = -0,1893$
 $w_{13} = 0,0562$
 $b \text{ (sesgo)} = -0,0031$

Son números chicos, cercanos a cero, positivos y negativos mezclados. **Nadie eligió esos valores a mano**. Los generó el programa al azar dentro de una distribución específica.

8.2. ¿Por qué al azar y no en cero?

Podrías pensar: “¿Y si arrancamos todos los pesos en cero, así partimos de un lugar neutro?”

Mala idea. Si todas las neuronas de una capa empiezan con los mismos pesos, **todas aprenden lo mismo**. Es como tener un aula donde todos los alumnos son gemelos idénticos: aprenden en paralelo pero nunca se especializan. La **asimetría inicial** (que cada neurona empiece con valores distintos) es lo que permite que después se especialicen en detectar cosas diferentes.

8.3. Cómo cambian los pesos: descenso por gradiente

Los pesos se ajustan mediante **backpropagation** usando un algoritmo llamado **descenso por gradiente**. La idea, en palabras simples:

- Paso 1.** Le mostrás a la red un ejemplo. Ella predice algo. Como los pesos son aleatorios al principio, va a predecir muy mal.
- Paso 2.** Calculás el **error**: qué tan lejos estuvo la predicción de la respuesta correcta.
- Paso 3.** El algoritmo se pregunta, para cada peso: “Si moviera este peso un poquito para arriba, ¿el error aumentaría o bajaría? ¿Y para abajo?”
- Paso 4.** Se mueve cada peso en la dirección que reduce el error. Solo un poquito.

Paso 5. Se repite con otro ejemplo. Y otro. Y otros mil millones.

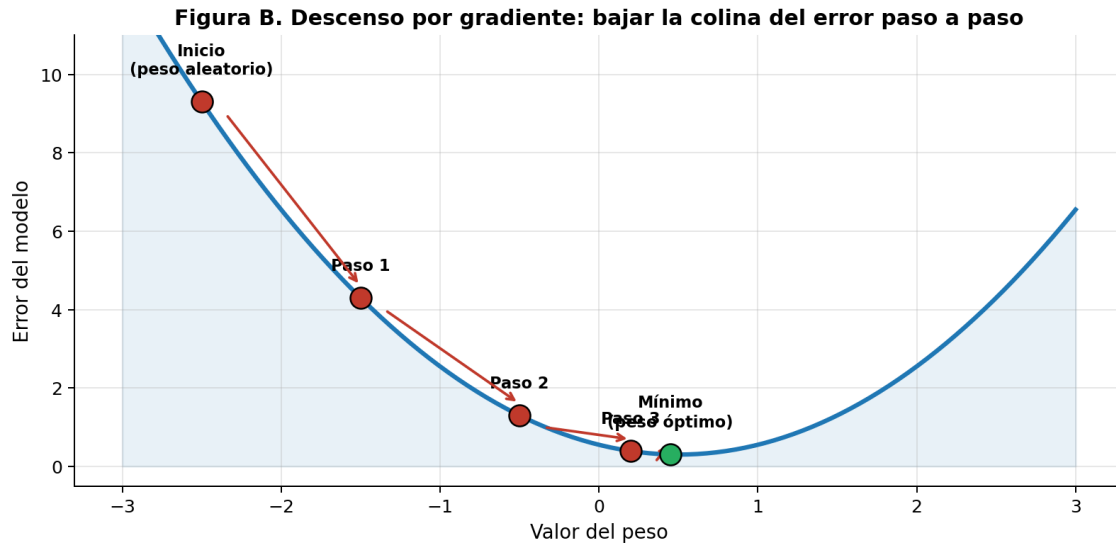


Figura B. Descenso por gradiente: bajar la colina del error paso a paso.

Ejemplo numérico paso a paso

Imaginá una red con un solo peso, w . Empieza en $w = 0,5$. La respuesta correcta es 10.

Iteración 1: con $w = 0,5$, la red predice 3. Error = $10 - 3 = 7$. El algoritmo detecta que subiendo w se reduce el error. Sube un poquito: w pasa a 0,6.

Iteración 2: con $w = 0,6$, predice 4. Error = 6. Se sigue subiendo: $w = 0,7$.

Iteración 3: con $w = 0,7$, predice 5. Error = 5. $w = 0,85$.

... y así, iteración tras iteración, hasta que w llegue a un valor donde predice cerca de 10.

Un LLM real hace **exactamente esto**, pero simultáneamente con miles de millones de pesos y billones de ejemplos.

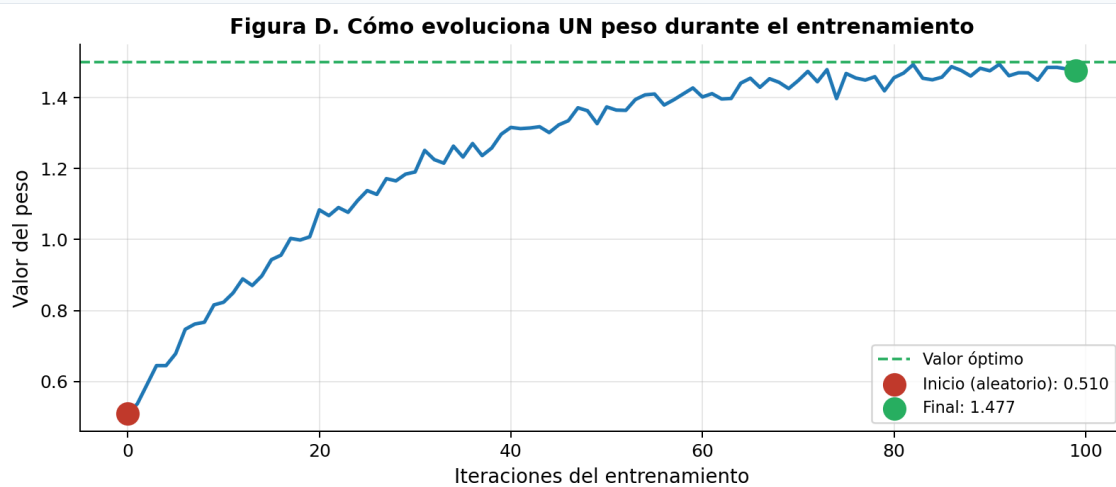


Figura D. Un solo peso: arranca aleatorio y converge hacia su valor útil.

8.4. Cuánto se mueve cada peso: la tasa de aprendizaje

Hay un parámetro clave llamado **learning rate** (tasa de aprendizaje), típicamente un número muy pequeño como 0,001 o 0,0001. Es cuánto se mueve cada peso en cada paso.

- Si es **muy grande**, los pesos brincan salvajemente y la red no converge.
- Si es **muy chica**, el entrenamiento es tan lento que llevaría años.

Elegir la tasa de aprendizaje correcta es parte del arte de entrenar modelos.

8.5. El resultado final

Después de entrenar durante meses, los pesos de un LLM son **números muy específicos** que nadie escribió a mano. Por ejemplo, un peso cualquiera podría terminar valiendo algo como -0,0387452 o 0,19234. Son números que salieron del proceso, no de una decisión humana.

Y acá viene la parte fascinante: **si abrís un LLM y mirás cualquier peso individual, no significa nada por sí solo**. Es un número anónimo. Pero el conjunto de los miles de millones de pesos, tomados como un todo, codifica todo lo que el modelo “sabe”: gramática, hechos, estilos, razonamientos. La inteligencia (por llamarla así) está **distribuida** entre todos ellos.

9. Las funciones de activación: quién las define

Acá la respuesta es distinta: **las funciones de activación las elige el humano que diseña la red**, no se aprenden.

Recordá que cada neurona hace dos cosas:

- **Suma sus entradas ponderadas por los pesos** (esto sí se aprende).
- **Pasa esa suma por una función de activación** (esto lo elige el diseñador).

Es como decidir con qué tipo de “válvula” se abre o cierra la neurona. Las funciones más comunes se ven así:

Figura A. Las funciones de activación más usadas

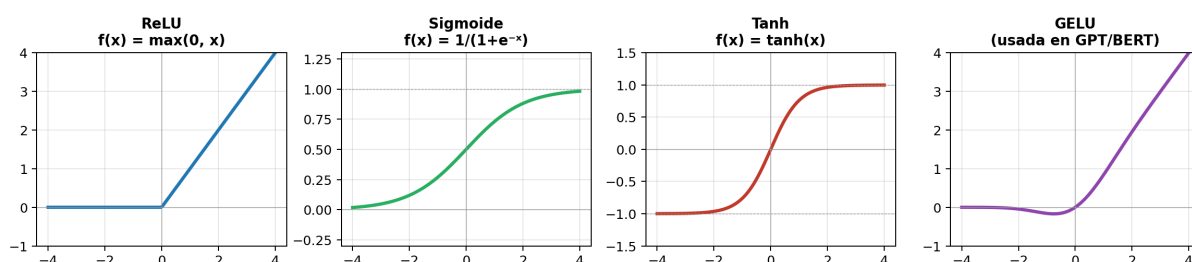


Figura A. Las funciones de activación más usadas.

9.1. Las funciones más comunes

ReLU (Rectified Linear Unit) — la más popular hoy

Fórmula ultra simple: $f(x) = \max(0, x)$.

- Si la suma es positiva, la deja pasar tal cual.
- Si es negativa, la aplasta a cero.

Simple, rápida de calcular, y funciona sorprendentemente bien. Es la reina en la mayoría de las redes modernas, incluidos los Transformers.

Sigmoide

Fórmula: $f(x) = 1 / (1 + e^{-x})$. Aplasta cualquier número entre 0 y 1. Se usa cuando querés interpretar la salida como una probabilidad. Fue muy popular antes; hoy menos, porque tiene problemas cuando la red es muy profunda.

Tanh (tangente hiperbólica)

Aplasta entre -1 y $+1$. Parecida a la sigmoide pero centrada en cero.

GELU, SiLU, Swish — las modernas

Variaciones más suaves de ReLU que se usan mucho en LLMs actuales. **GELU** es la que usa GPT, por ejemplo. Son curvas parecidas a ReLU pero con transiciones suaves cerca del cero.

9.2. Cómo se elige cuál usar

Es una decisión de **arquitectura** que toma el equipo que diseña el modelo, basándose en:

- **Experimentación**: se prueban varias y se ve cuál funciona mejor.

- **Estado del arte:** hay convenciones. Para redes convolucionales de visión, ReLU. Para Transformers modernos, GELU. Para la capa final de un clasificador binario, sigmoide.
- **Investigación previa:** papers académicos que muestran qué funciona mejor en qué caso.

Una vez que se elige, **esa función queda fija durante todo el entrenamiento**. No cambia. Lo que cambia son los pesos que alimentan a la neurona.

10. Reparto humano / máquina

Para dejarlo cristalino, este es el reparto entre **lo que decide el humano** y **lo que aprende la máquina**:

Figura C. Reparto de tareas: humano vs. máquina

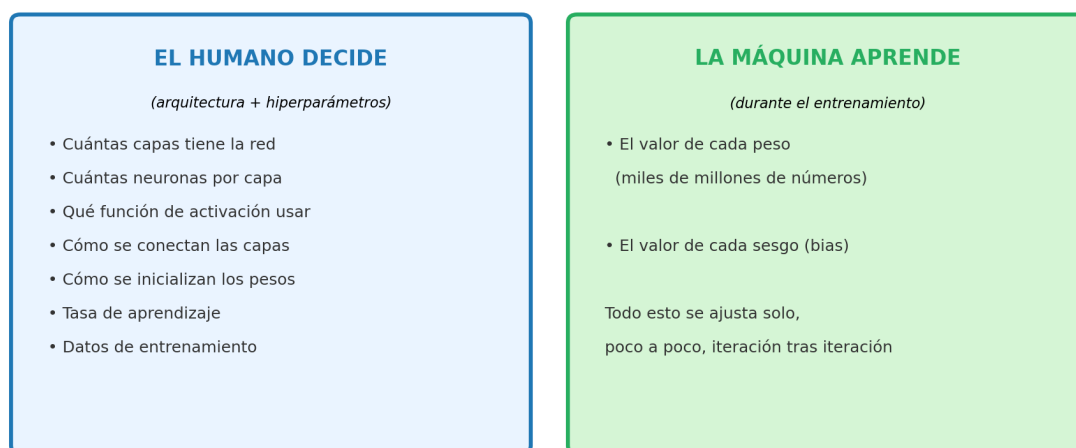


Figura C. División de tareas en el diseño y entrenamiento de una red neuronal.

Es una división de trabajo interesante: **el humano diseña el molde vacío; los datos llenan el molde con valores concretos.**

11. Analogía final: el edificio de las perillas

Imaginá un edificio con miles de millones de **perillas de volumen**. El arquitecto (el humano) decidió:

- Cuántas perillas hay.
- Cómo están conectadas entre sí.
- Qué tipo de perilla es cada una (algunas cortan, otras filtran, otras amplifican).

Pero la **posición exacta de cada perilla** (qué tan giradas están) no la puso el arquitecto. Se ajustan solas, poco a poco, durante meses, cada vez que el edificio “escucha” un texto y trata de adivinar la siguiente palabra. Al final, esas millones de perillas están en las posiciones perfectas para reproducir lenguaje humano.

Nadie sabe exactamente qué hace cada perilla individual. Pero el conjunto funciona.

PARTE IV

Síntesis final y glosario rápido

12. Síntesis del manual

Todo lo visto se puede resumir en cinco ideas nucleares:

1. Qué predice un LLM

No predice tu prompt. Predice el **siguiente token** que viene DESPUÉS de todo el texto acumulado hasta ese momento. La respuesta se construye token a token (generación autorregresiva).

2. Qué significa que “entiende”

Es una **metáfora**. No hay comprensión consciente: hay un patrón estadístico gigantesco aprendido de textos humanos que se comporta funcionalmente como si entendiera.

3. Cómo se entrena

En **tres fases**:

- **Pre-entrenamiento**: leer internet entero prediciendo la próxima palabra.
- **SFT**: ajustar con miles de conversaciones ideales.
- **RLHF**: refinar con retroalimentación humana.

En cada fase se ajustan los mismos pesos, con distintos datos y objetivos.

4. Qué son los pesos

Números que **arrancan aleatorios** y se ajustan durante el entrenamiento mediante **descenso por gradiente**. Un LLM grande tiene miles de millones de ellos. Ningún peso individual “significa” algo: el conocimiento está **distribuido** en el conjunto.

5. Qué decide el humano vs. qué aprende la máquina

El humano diseña la **arquitectura** (cuántas capas, cuántas neuronas, qué funciones de activación, cómo se conectan) y los **hiperparámetros** (tasa de aprendizaje, método de inicialización). La máquina aprende los **valores concretos** de todos los pesos y sesgos.

13. Glosario rápido de este manual

Predicción autorregresiva	Generar texto token a token, usando lo generado como parte de la nueva entrada.
Token	Unidad mínima en la que se divide el texto para procesarlo.
Peso	Número interno de la red que se ajusta durante el entrenamiento.
Sesgo (bias)	Término independiente que cada neurona suma junto con las entradas ponderadas.
Inicialización	Manera de asignar los valores aleatorios iniciales a los pesos (Xavier, He, etc.).
Backpropagation	Algoritmo para calcular hacia atrás cuánto contribuyó cada peso al error.

Descenso de gradiente	por	Estrategia de ajuste: mover cada peso en la dirección que reduce el error.
Tasa de aprendizaje		Cuánto se mueve cada peso en cada paso. Típicamente muy pequeña (0,001).
Función de activación		Función que decide qué sale de la neurona. La elige el humano al diseñar.
ReLU / GELU / Sigmoid		Funciones de activación más comunes hoy en Deep Learning.
Pre-entrenamiento		Fase 1: leer masivamente texto y aprender a predecir el siguiente token.
Fine-tuning (SFT)		Fase 2: ajustar el modelo con ejemplos de conversaciones ideales.
RLHF		Fase 3: refinamiento con retroalimentación humana para hacerlo útil y seguro.
Modelo de recompensa	de	Red aparte que aprende a puntuar respuestas según preferencias humanas.
Hiperparámetro		Ajuste del entrenamiento decidido por el humano (no aprendido).
Arquitectura		Diseño estructural de la red: capas, conexiones, tipos de neuronas.

— Fin del manual —

Este manual complementa los materiales anteriores. Vuelve a él cuando necesites recordar exactamente qué predice un LLM, cómo se entrena, y qué es lo humano y qué es lo aprendido.